=== UNIT 1: PYTHON FUNDAMENTALS, ENVIRONMENT & SYNTAX ===

>> 1.1 Evolution, Philosophy and Characteristics of Python

Python is an interpreted, high-level, dynamically-typed, multi-paradigm programming language conceived by Guido van Rossum in 1989 and officially released in 1991. It emphasizes human code readability, rapid application prototyping, and clean expressive syntax encapsulated in 'The Zen of Python' (PEP 20).
* Execution Architecture: Unlike pure compiled languages (such as C/C++) that compile directly to native machine code, Python source code (.py) is compiled by the CPython virtual machine into intermediate Bytecode (.pyc) stored inside the __pycache__ directory. The Python Virtual Machine (PVM) then interprets and executes this bytecode at runtime.
* Distinctive Features: (1) Dynamic Typing: Variable data types are inferred and bound at runtime rather than compile-time. (2) Automatic Memory Management: Built-in reference counting supplemented by a cyclic generational Garbage Collector (GC). (3) Indentation-Delimited Block Structure: Eliminates curly braces {} and begin/end tokens, using strict 4-space whitespace indentation (PEP 8). (4) Extensive Standard Library: 'Batteries Included' philosophy providing comprehensive modules for I/O, networking, regular expressions, and mathematics.

>> 1.2 Python Interactive Shells, IDEs and Package Management

* Interactive REPL (Read-Eval-Print Loop): Allows instant line-by-line expression evaluation and interactive experimentation.
* Integrated Development Environments (IDEs): VS Code, PyCharm, and Jupyter Notebooks facilitate rich syntax highlighting, integrated debugging, and virtual environment switching.
* Package Management via PIP and Virtual Environments: The pip package installer fetches third-party libraries from the Python Package Index (PyPI). Virtual environments (python -m venv env) isolate project dependencies, preventing version collisions across distinct applications.

>> 1.3 Variables, Dynamic Typing and Memory Allocation

* In Python, variables are NOT typed memory containers; they are symbolic name references (pointers) bound to objects residing in heap memory.
* Object Model: Every Python entity is a first-class object containing three

fundamental attributes: (1) Identity: Memory address retrieved via id(x). (2) Type: Object classification retrieved via type(x). (3) Value: Internal state data.
* Mutable vs Immutable Objects: Immutable types cannot have their in-place state altered post-instantiation (int, float, complex, bool, str, tuple, bytes, frozenset). Mutable types permit direct in-place modification (list, dict, set, bytearray).
* Small Integer Caching (-5 to 256): CPython pre-allocates and caches integer objects in this range, meaning separate assignments bind to the identical memory address.

>> 1.4 Fundamental Data Types and Type Conversions

* Numeric Types: int (arbitrary precision integers without overflow limits), float (64-bit IEEE 754 double precision floating-point), complex (rectangular representation z = a + bj).
* Boolean Type: bool (subclass of int, having values True (1) and False (0)).
* Type Casting: Implicit type coercion (e.g. adding int + float yields float) and Explicit type conversion functions (int(), float(), str(), bool(), list(), tuple()).

>> 1.5 Operators and Operator Precedence in Python

* Arithmetic Operators: Addition (+), Subtraction (-), Multiplication (*), True Division (/ yields float), Floor Division (// truncates to floor integer), Modulus (%), Exponentiation (**).
* Comparison (Relational) Operators: ==, !=, >, <, >=, <=, and Chained Comparisons (e.g., 0 < x <= 100).
* Logical Operators: and, or, not (utilizing Short-Circuit Evaluation: expr1 and expr2 evaluates expr2 only if expr1 is truthy).
* Bitwise Operators: &, |, ^, ~, <<, >> operating on binary bit representations.
* Identity vs Equality Operators: is checks memory identity (id(a) == id(b)); == checks value equality.
* Membership Operators: in and not in test presence of element within sequence/collection.

>> 1.6 Standard Console Input, Output and String Formatting

* Console Input: The input(prompt) built-in function halts program execution, reads a line of input from stdin as a string, and strips trailing newlines.
* Console Output: print(*objects, sep=' ', end='\\n', file=sys.stdout, flush=False).
* String Formatting Paradigms: (1) Old-style printf formatting (%s, %d, %f), (2) str.format() method, (3) Formatted String Literals (F-Strings, introduced in Python

3.6): f'{var:.2f}'.

[PROGRAM 1: Interactive BMI Health Calculator with F-Strings]

Problem Context: Develop a robust Python console application that reads user name, weight in kilograms, and height in centimeters. Compute Body Mass Index (BMI = weight / (height_in_m)^2) and display a categorized report formatted to 2 decimal places using modern f-strings.

Python Source Code:

```
# BMI Health Assessment Application
def calculate_bmi():
name = input('Enter client name: ').strip()
weight_kg = float(input('Enter body weight (kg): '))
height_cm = float(input('Enter height (cm): '))

height_m = height_cm / 100.0
bmi = weight_kg / (height_m ** 2)

if bmi < 18.5:
category = 'Underweight'
elif bmi < 24.9:
category = 'Normal weight'
elif bmi < 29.9:
category = 'Overweight'
else:
category = 'Obesity'

print(f'\\n--- Health Assessment Report for {name.upper()} ---')
print(f'Weight: {weight_kg:.1f} kg | Height: {height_m:.2f} m')
print(f'Computed BMI: {bmi:.2f} kg/m^2')
print(f'Diagnostic Category: {category}')
```

Execution Trace & Output:

Input: Name = 'Aarav', Weight = 72.5, Height = 175.0

height_m = 1.75; bmi = 72.5 / (1.75 ** 2) = 72.5 / 3.0625 = 23.67 kg/m^2.

Output: 'Computed BMI: 23.67 kg/m^2 | Diagnostic Category: Normal weight'.

[PROGRAM 2: Bitwise Arithmetic and Network Subnet Masking]

Problem Context: Implement an IP network utility in Python that reads an IPv4 address and subnet mask as 32-bit integers, performs bitwise AND masking to determine the network address, and calculates total usable host addresses.

Python Source Code:

```
# Bitwise IP Network Masking Utility
def ip_to_int(octet1, octet2, octet3, octet4):
return (octet1 << 24) | (octet2 << 16) | (octet3 << 8) | octet4

def int_to_ip(ip_int):
return f'{(ip_int >> 24) & 255}.{(ip_int >> 16) & 255}.{(ip_int >> 8) & 255}.{ip_int &
255}'

ip_val = ip_to_int(192, 168, 1, 145)      # 192.168.1.145
mask_val = ip_to_int(255, 255, 255, 224)  # /27 Subnet (255.255.255.224)
network_val = ip_val & mask_val
broadcast_val = network_val | (~mask_val & 0xFFFFFFFF)
usable_hosts = (broadcast_val - network_val) - 1

print(f'Host IP Address:      {int_to_ip(ip_val)}')
print(f'Subnet Mask:          {int_to_ip(mask_val)}')
print(f'Network Address:      {int_to_ip(network_val)}')
print(f'Broadcast Address:    {int_to_ip(broadcast_val)}')
print(f'Usable Host Capacity: {usable_hosts}')
```

Execution Trace & Output:

IP: 192.168.1.145 & 255.255.255.224 => 145 (10010001) & 224 (11100000) = 128 (10000000).
Network Address = 192.168.1.128; Broadcast = 192.168.1.159; Usable hosts = 30.

[PROGRAM 3: Variable Identity, Memory References and Mutability]

Problem Context: Write a program demonstrating the difference between memory identity (is) and value equality (==) for mutable vs immutable objects, and verify CPython small integer caching.

Python Source Code:

```
# Exploring Python Object Model & Memory Caching
# Case 1: Small Integer Caching (-5 to 256)
a = 250; b = 250
print(f'a is b (250): {a is b} | id(a): {id(a)}, id(b): {id(b)}')

# Case 2: Large Integers outside cache range
x = 1000; y = 1000
```

```
print(f'x == y: {x == y} | x is y: {x is y}')

# Case 3: Mutable Lists
list1 = [1, 2, 3]; list2 = [1, 2, 3]
print(f'list1 == list2: {list1 == list2} | list1 is list2: {list1 is list2}')

# Case 4: Aliased Reference Mutation
list3 = list1
list3.append(4)
print(f'After mutating list3: list1 = {list1} | list1 is list3: {list1 is list3}')
```

Execution Trace & Output:

a is b (250): True (identical memory address due to integer interning).

x is y (1000): False (separate heap allocations).

list1 is list2: False; list1 is list3: True; list1 becomes [1, 2, 3, 4] due to shared pointer aliasing.

[PROGRAM 4: Compound Interest and Investment Horizon Table]

Problem Context: Write a program calculating compound growth of a principal investment P with annual interest rate r compounded n times per year over t years using formula A = P * (1 + r/n)**(n*t). Generate a formatted financial tabular report.

Python Source Code:

```
# Compound Interest Growth Modeler
principal = 50000.0  # INR 50,000
rate = 0.08          # 8.0% annual interest
compounds_per_year = 4  # Quarterly compounding

print(f'Initial Principal: INR {principal:,.2f} | Annual Rate: {rate*100:.1f}%')
print('Year | Total Balance (INR) | Interest Earned (INR)')
print('-' * 50)
for year in range(1, 6):
balance = principal * (1 + rate / compounds_per_year) ** (compounds_per_year * year)
interest = balance - principal
print(f'{year:4d} | {balance:19,.2f} | {interest:21,.2f}')
```

Execution Trace & Output:

Year 1: Balance = INR 54,121.61 | Interest = INR 4,121.61

Year 5: Balance = INR 74,297.37 | Interest = INR 24,297.37

>> 1.7 University Review & Examination Questions

* Q1: Explain Python’s memory management architecture, distinguishing between reference counting and generational garbage collection.
Answer: CPython manages memory primarily through Reference Counting. Every Python object header contains an ob_refcnt counter tracking how many references point to it. When an assignment, function call, or list insertion binds to the object, ob_refcnt increments; when a variable goes out of scope or is reassigned, ob_refcnt decrements. When ob_refcnt drops to zero, memory is deallocated immediately. However, reference counting fails to reclaim Circular References (e.g. object A references B, and B references A). To resolve this, CPython incorporates a Generational Garbage Collector that segregates objects into three generations (Gen 0, 1, 2) based on survival time and periodically sweeps through pointers to break unreachable circular reference cycles.
* Q2: Differentiate between the ’==’ operator and the ’is’ operator in Python with concrete illustrative examples.
Answer: The ’==’ operator evaluates Value Equality: it calls the __eq__() dunder method to check whether two distinct objects hold equivalent data content. The ’is’ operator evaluates Object Identity: it compares the raw memory addresses of two operands (id(a) == id(b)) to check if both variable names point to the exact same physical heap object. For example, if a = [1, 2] and b = [1, 2], a == b evaluates to True because their element values match, but a is b evaluates to False because two independent list objects were allocated at distinct memory locations.

>> 1.8 Key Takeaways and Revision Points

* Python source code compiles to bytecode (.pyc) executed on the Python Virtual Machine (PVM).
* Variables are dynamic symbolic references, not statically typed storage boxes.
* Immutable types (int, float, str, tuple) cannot be altered in-place; mutable types (list, dict, set) can.
* F-strings (f’{var}’) offer superior performance and readability for formatted output.